

UNIVERSIDADE FEDERAL DO PARANÁ

LUCAS TAVARES ROCKEMBACH

DESMISTIFICAR – SENSOR DE POEIRA E FUMAÇA SHINYEI PPD42
RELATÓRIO

CURITIBA
2026



Introdução

Este relatório visa descrever os princípios de funcionamento do sensor de poeira e qualidade do ar Shinyei PPD42, bem como a explicação didática do firmware desenvolvido para leitura da qualidade do ar via Thingsboard e do circuito projetado para seu funcionamento, permitindo que o projeto possa ser replicado no futuro por qualquer um que tenha em mãos os componentes e o relatório. Vale ressaltar que todas as etapas de projeto de circuitos, instrumentação eletrônica e programação do microcontrolador foram desenvolvidas através do vínculo com o projeto de extensão Desmistificar do GICS, visando obter soluções tecnológicas eficientes e ecológicas.

Componentes Necessários

Os componentes eletrônicos necessários para a correta execução do projeto são os seguintes:

- Sensor de poeira Shinyei PPD42
- Kit de desenvolvimento ESP32
- Resistores de 10 K Ω e 47 K Ω
- Protoboard
- Jumpers

Princípios de Funcionamento

O sensor de poeira Shinyei PPD42 utiliza o princípio de dispersão da luz (efeito de espalhamento) para detectar partículas suspensas no ar. Em seu interior, um LED infravermelho emite luz através de uma região de detecção, enquanto um fotodetector mede a intensidade da luz espalhada pelas partículas que atravessam essa região. A presença de partículas provoca variações no sinal recebido pelo fotodetector, permitindo ao circuito interno identificar e contabilizar esses eventos. Dessa forma, a concentração de material particulado pode ser estimada a partir do tempo em que o sinal permanece em nível baixo (*Low Pulse Occupancy Time* – LPO), sendo esse princípio utilizado pelo PPD42 para estimar a quantidade de partículas presentes no ambiente.

Divisor de Tensão Projetado

Como é descrito na página 3 do datasheet presente no anexo, a tensão equivalente ao nível lógico alto do sinal do sensor é de 4 V, desse modo, como os pinos GPIO do ESP32 são projetados para lidarem com 3,3 V, optou-se por adicionar um divisor de tensão ao conectar o pino de sinal ao microcontrolador. Vale ressaltar, que, para um teste inicial, é provável que o circuito funcione normalmente sem o divisor, devido ao fato de as duas tensões serem relativamente próximas em valor, mas, levando em consideração a durabilidade e segurança à longo prazo dos pinos GPIO do Esp32, recomenda-se o uso dos resistores adicionais.

Nesse viés, os valores dos resistores foram estabelecidos em $R_1 = 10\text{ K}\Omega$ e $R_2 = 47\text{ K}\Omega$ para que a tensão do pino D4 do Esp32 seja de 3,3 V ao considerar que a tensão no pino de saída do sinal do sensor é 4V. Fato este que pode ser demonstrado pela equação 1:

$$V_{D4} = V_{\text{sensor}} \frac{R_2}{R_1 + R_2} = 4 \frac{47\text{ k}\Omega}{10\text{ k}\Omega + 47\text{ k}\Omega} \approx 3,3\text{ V}$$

Equação 1: resistências para o divisor de tensão.

Sendo assim, foram estabelecidas as seguintes conexões:

- Pino 1 -> GND
- Pino 2 -> Divisor de tensão -> Pino D4 do ESP32
- Pino 3 -> +5 VCC

Que podem ser visualizadas através do esquemático da figura 1, feito no software Proteus:

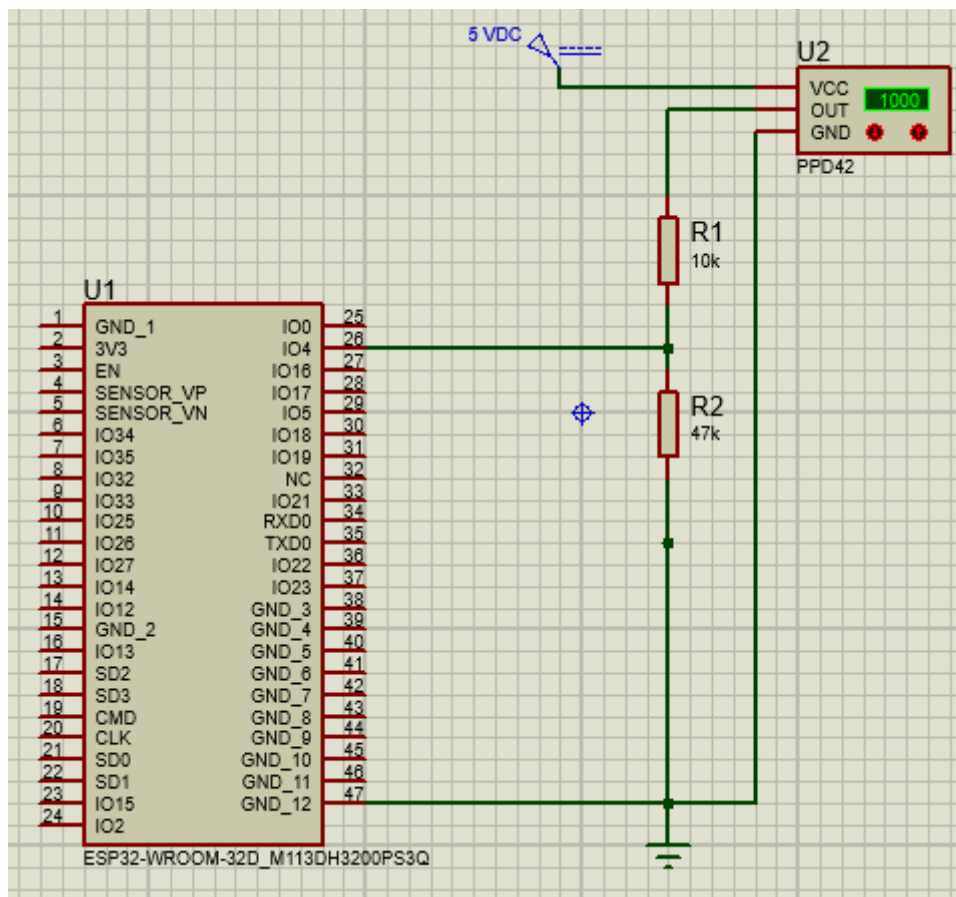


Figura 1: esquemático das conexões.

Adicionalmente, para possibilitar a alimentação de 5 V, é recomendado alimentar o ESP32 com o computador via USB e então conectar o pino “5V” do DEVKIT ao pino “VCC” do sensor.

Firmware

Segue abaixo o código do firmware desenvolvido para o projeto, com comentários descritos de forma didática:

```
Desmistificar_eduroam.ino
1 //Bibliotecas utilizadas
2 #include <Arduino.h>
3 #include <WiFi.h>
4 #include <esp_wpa2.h>
5 #include <Arduino_MQTT_Client.h>
6 #include <ThingsBoard.h>
7
8 // Configurações do ThingsBoard
9 #define THINGSBOARD_SERVER "192.168.1.1"
10 #define THINGSBOARD_TOKEN "12345678901234567890"
11 #define THINGSBOARD_PORT 1883 // define a porta do thingsboard
12
13 // Tempo de amostragem de 30 s
14 #define SAMPLE_TIME_MS 30000
15
16 // Wi-Fi eduroam
17 const char* ssid = "eduroam";
18 const char* username = ""; //preencher
19 const char* password = ""; //preencher
20
21 // Pino de saída de dados do sensor PPD42 conectado ao pino D4 do Esp32
22 int sensor_pin = 4;
23 volatile uint64_t t_baixo = 0; // tempo acumulado em que o sinal do sensor permanece em nível lógico baixo
24 volatile uint64_t t_start = 0; // instante em que se inicia um pulso em nível lógico baixo
25
```

Figura 2: firmware desenvolvido.

```
Desmistificar_eduroam.ino
26 // Variáveis de cálculo
27 unsigned long lastTime = 0;
28 unsigned long ocup_baixo = 0;
29 float razao = 0;
30 float concentracao = 0;
31
32 // WiFi e MQTT
33 WiFiClient espClient;
34 Arduino_MQTT_Client mqttClient(espClient);
35
36 // Define o tamanho máximo da mensagem (1 KB é suficiente)
37 constexpr uint32_t MAX_MESSAGE_SIZE = 1024U;
38
39 // Inicializa ThingsBoard no formato moderno
40 ThingsBoard tbClient(mqttClient, MAX_MESSAGE_SIZE);
41
42 // Função de interrupção para medições
43 void IRAM_ATTR pulso_in() {
44     unsigned long agora = micros();
45     if (digitalRead(sensor_pin) == LOW) {
46         t_start = agora;
47     } else if ((agora - t_start) > 100) {
48         t_baixo += (agora - t_start);
49         t_start = 0;
50     }
51 }
```

Figura 3: firmware desenvolvido.

```

Desmistificar_eduroam.ino
52
53 void setup() {
54   Serial.begin(115200);
55
56   // Configurações para conectar ao Wi-Fi
57   WiFi.disconnect(true);
58   WiFi.mode(WIFI_STA);
59   esp_wifi_sta_wpa2_ent_enable();
60   esp_wifi_sta_wpa2_ent_set_identity((uint8_t *)username, strlen(username));
61   esp_wifi_sta_wpa2_ent_set_username((uint8_t *)username, strlen(username));
62   esp_wifi_sta_wpa2_ent_set_password((uint8_t *)password, strlen(password));
63   WiFi.begin(ssid);
64   while (WiFi.status() != WL_CONNECTED) {
65     delay(1000);
66     Serial.println("Conectando ao Wi-Fi...");
67   }
68   Serial.println("Conectado ao Wi-Fi!");
69   Serial.print("Conectando ao Wi-Fi");
70
71   // Configurar MQTT para o PubSubClient
72   mqttClient.set_server(THINGSBOARD_SERVER, THINGSBOARD_PORT); //Caso o código não funcione de primeira, comentar essa linha e tentar novamente
73
74   // Define o pino do sensor conectado ao Esp32 como entrada de dados, pois a medição sai do medidor e entra no microcontrolador
75   pinMode(sensor_pin, INPUT);
76   attachInterrupt(digitalPinToInterrupt(sensor_pin), pulso_in, CHANGE); // atribui a função de interrupção ao pino com a entrada de dados do sensor,
77   //quando o nível lógico do sinal medido muda
78 }
79

```

Figura 4: firmware desenvolvido.

```

Desmistificar_eduroam.ino
80 // Função para reconectar ao Thingsboard caso a conexão inicial dê errado
81 void reconnectTB() {
82   while (!tbClient.connected()) {
83     Serial.println("Conectando ao ThingsBoard...");
84
85     // CORREÇÃO: Conectar usando o método correto
86     if (tbClient.connect(THINGSBOARD_SERVER, THINGSBOARD_TOKEN)) {
87       Serial.println("Conectado ao ThingsBoard!");
88     } else {
89       Serial.print("Falha na conexão! Código: ");
90     }
91   }
92 }
93

```

Figura 5: firmware desenvolvido.


```

Desmistificar_educomino
94 void loop() {
95
96     // Caso ainda não tenha conectado ao thingsboard, tenta reconectar
97     if (!tbClient.connected()) {
98         reconnectTB();
99     }
100
101     // Chama de maneira iterativa a função de conexão do cliente MQTT do thingsboard
102     tbClient.loop();
103
104     // Realiza as medições dentro do tempo de amostragem
105     unsigned long agora = millis();
106     if (agora - lastTime >= SAMPLE_TIME_MS) {
107         lastTime = agora;
108
109         noInterrupts(); // desabilita as interrupções
110         ocup_baixo = t_baixo; // registra o tempo em que a leitura permaneceu em nível lógico baixo
111         t_baixo = 0; // zera o tempo em nível lógico baixo para permitir que a operação seja repetida iterativamente
112         interrupts(); // habilita a função de interrupção para realizar as medições
113
114         // Aplica a equação de calibração do datasheet para obter a concentração de partículas de poeira em pcs/0.01cf a partir do tempo em que o sensor permaneceu em nível lógico baixo
115         razao = ocup_baixo / (SAMPLE_TIME_MS * 10.0);
116         concentracao = 1.1 * pow(razao, 3) - 3.8 * pow(razao, 2) + 520 * razao + 0.62;
117
118         // Exibe os resultados no monitor serial para auxiliar na interpretação e depuração do sistema, mais utilizado para debug
119         Serial.print("Razão = ");
120         Serial.print(razao);
121         Serial.print(" | Concentração = ");
122         Serial.print(concentracao);
123         Serial.println(" pcs/0.01cf");
124
125         // Envia dados de telemetria
126         tbClient.sendTelemetryData("concentracao", concentracao);
127     }
128
129     // Espera 100 ms para repetir a medição
130     delay(100);
131 }

```

Figura 6: firmware desenvolvido.

O código foi desenvolvido intencionalmente com comentários explicativos para facilitar o entendimento, mas ainda é destacável que na linha 76 (figura 4), a interrupção precisa ser configurada como “CHANGE” para que consiga detectar o instante de mudança do nível lógico alto para baixo e iniciar a medição e processamento de dados das partículas de poeira de forma sincronizada, e também a mudança de baixo para alto, encerrando a medição.

Além disso, a equação de calibração nas linhas 114-116 (figura 6) do laço principal, foi obtida através de uma aproximação polinomial da curva do tempo percentual de ocupação do sinal em nível lógico baixo, em função da concentração de partículas (figura 7) encontrada na página 3 do datasheet anexado, e seus coeficientes podem ser submetidos a ajustes finos, caso melhorias na precisão sejam necessárias. Ademais, a concentração obtida pela curva é expressa em partículas por 0,01 pés-cúbicos, conforme a especificação do sensor, assim, caso seja desejada uma representação em unidades do Sistema Internacional, é necessário realizar a conversão apropriada da unidade de volume, sem alterar a grandeza física medida.

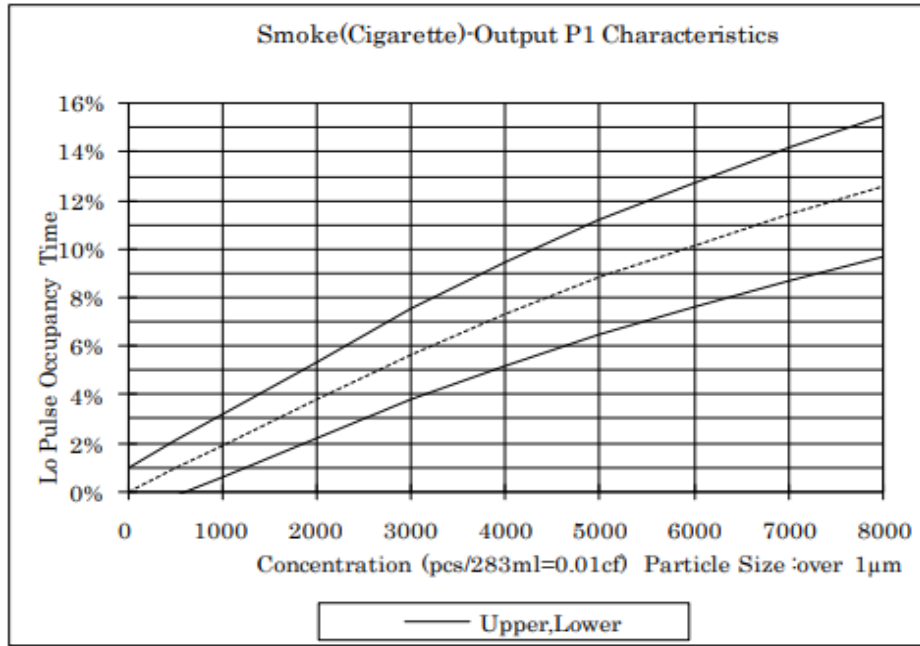


Figura 7: curva do datasheet.

A aproximação polinomial foi feita através das equações 2 e 3, calculando a razão percentual do tempo de nível lógico baixo, e relaciona ela com a concentração de partículas de poeira:

$$R = \frac{T_{LOW}}{T_{amostra} \times 10}$$

Equação 2: razão percentual do tempo de ocupação em nível lógico baixo.

$$C = 1,1R^3 - 3,8R^2 + 520R + 0,62$$

Equação 3: aproximação polinomial que relaciona a razão com a concentração de partículas em pcs/0.01cf.

Conclusão

Em suma, o projeto reúne um conjunto de soluções em Hardware e Software/Firmware que trabalham conceitos de instrumentação eletrônica, projeto de circuitos elétricos, programação e ecologia para permitir a medição da concentração de partículas de poeira no ar. Por fim, uma possível próxima etapa sugerida, seria a confecção de uma placa de circuito impresso (PCB) com o hardware desenvolvido para que o projeto seja levado para outro nível.

Anexo

Datasheet disponível em:

<https://drive.google.com/drive/u/0/folders/13B9tggSgA8fl50nbsg4WqPF0bv9A-sOq>